UF2405. UD6. Actividad 1_Caso práctico "APLICACIÓN CON ACCESO A UNA BASE DE DATOS RELACIONAL DESDE UNA CLASE"

Proyecto práctico: Gestor de contactos con Java y MySQL

Memoria técnica y resolución de actividades
Versión 1.0 · Octubre de 2026

1. Presentación del proyecto

La mejor forma de presentar el proyecto Proyecto_UF2405_Codigo.zip es mostrandolo como muestra práctica de desarrollo Java con acceso a una base de datos relacional, acompañado de esta memoria, que resuelve las once cuestiones de la actividad.

El proyecto contiene una aplicación web Java con Maven, un modelo Contacto, un DAO (ContactoDAO), un servlet controlador y una interfaz JSP. También incorpora configuración para MySQL mediante Docker.

Distinción importante: el proyecto demuestra el acceso a la base de datos desde clases Java, pero utiliza una arquitectura MVC y no implementa necesariamente todos los cambios concretos del enunciado. Se conserva el proyecto original y se documentan las modificaciones necesarias, sin afirmar que se han ejecutado pruebas que aún no se hayan realizado.

1.1. Tecnologías y archivos principales

Elemento	Función
Java 11	Lenguaje de desarrollo.
JDBC	Conectividad entre Java y MySQL.
MySQL 8	Sistema gestor de base de datos relacional.
Maven	Gestión de dependencias y construcción.
Servlets y JSP	Controlador de peticiones e interfaz web.
Docker	Configuración del entorno de ejecución.
Contacto.java	Modelo de datos de los contactos.
ContactoDAO.java	Acceso a datos y operaciones SQL.
ContactoServlet.java	Gestión de peticiones de la aplicación.
index.jsp	Presentación de contactos y formulario de alta.
pom.xml / docker-compose.yml	Dependencias y entorno de ejecución.

1.2. Texto de presentación

El proyecto consiste en una aplicación web desarrollada en Java que permite consultar y registrar contactos almacenados en una base de datos relacional MySQL. La aplicación utiliza una arquitectura organizada en capas: Contacto representa cada registro; ContactoDAO centraliza el acceso a la base de datos mediante JDBC; ContactoServlet actúa como controlador, y la vista JSP presenta los datos al usuario.

La actividad aborda la creación y modificación de tablas, inserción de registros, relaciones entre tablas, restricciones e índices, así como la modificación de una clase Java para filtrar contactos por edad y la planificación de pruebas.

2. Resolución de las cuestiones SQL

Las consultas siguientes están preparadas para MySQL 8. Se recomienda ejecutarlas en una base de datos de pruebas.

Actividad 1. Crear la tabla CONTACTOS

```
CREATE TABLE CONTACTOS (
   Id INT AUTO_INCREMENT PRIMARY KEY,
   Nombre VARCHAR(50) NOT NULL,
   Apellidos VARCHAR(100),
   Edad INT
);
```

Id es un entero autoincremental y clave primaria. Nombre es obligatorio mediante NOT NULL. Apellidos admite hasta 100 caracteres y Edad almacena un número entero.

Actividad 2. Insertar los contactos indicados

```
INSERT INTO CONTACTOS (Nombre, Apellidos, Edad)
VALUES
   ('Alejandro', 'Pérez López', 35),
   ('Raimundo', 'Guzmán Álvarez', 30);

SELECT * FROM CONTACTOS;
```

Id	Nombre	Apellidos	Edad
1	Alejandro	Pérez López	35
2	Raimundo	Guzmán Álvarez	30

Los identificadores serán 1 y 2 si la tabla es nueva y no hay inserciones previas.

Actividad 3. Añadir la columna Direccion

```
ALTER TABLE CONTACTOS
ADD COLUMN Direccion VARCHAR(150);
```

ALTER TABLE modifica la estructura de una tabla. La columna nueva admite NULL porque no se declara NOT NULL.

Actividad 4. Crear y rellenar la tabla ACTIVIDADES

```
CREATE TABLE ACTIVIDADES (
  Id INT AUTO_INCREMENT PRIMARY KEY,
  Actividad VARCHAR(100) NOT NULL
);

INSERT INTO ACTIVIDADES (Actividad)
VALUES
  ('Informático'),
  ('Consultor');

SELECT * FROM ACTIVIDADES;
```

Id	Actividad
1	Informático

Actividad 5. Añadir el campo Actividad a CONTACTOS

```
ALTER TABLE CONTACTOS
ADD COLUMN Actividad INT;
```

Este campo numérico almacenará el identificador de la actividad profesional. Se diferencia de ACTIVIDADES.Actividad, que contiene la descripción textual.

Actividad 6. Crear la relación entre las tablas

```
ALTER TABLE CONTACTOS
ADD CONSTRAINT fk_contactos_actividades
FOREIGN KEY (Actividad)
REFERENCES ACTIVIDADES(Id);

UPDATE CONTACTOS SET Actividad = 1 WHERE Id = 1;
UPDATE CONTACTOS SET Actividad = 2 WHERE Id = 2;

SELECT c.Id, c.Nombre, c.Apellidos, c.Edad,
    a.Actividad AS NombreActividad
FROM CONTACTOS c
LEFT JOIN ACTIVIDADES a ON c.Actividad = a.Id;
```

La clave foránea impide asignar un identificador de actividad inexistente. LEFT JOIN muestra también los contactos sin actividad asignada.

Actividad 7. Crear un índice único de dos formas

Forma 1: mediante CREATE UNIQUE INDEX.

```
CREATE UNIQUE INDEX idx_contactos_actividad
```

```
ON CONTACTOS (Actividad);

Forma 2: mediante ALTER TABLE.

ALTER TABLE CONTACTOS

ADD UNIQUE INDEX idx_contactos_actividad (Actividad);
```

Ambas sentencias crean un índice único. Esto impide repetir valores no nulos de Actividad en CONTACTOS; MySQL permite varios NULL en un índice único. Por tanto, cada actividad podría asignarse como máximo a un contacto. Si varios contactos deben compartir una actividad, no se debe crear este índice único.

Actividad 8. Eliminar el índice

```
DROP INDEX idx_contactos_actividad

ON CONTACTOS;
```

Elimina el índice sin borrar la columna Actividad.

Actividad 9. Eliminar la tabla ACTIVIDADES

```
ALTER TABLE CONTACTOS

DROP FOREIGN KEY fk_contactos_actividades;


DROP TABLE ACTIVIDADES;
```

La restricción se elimina primero porque CONTACTOS depende de ACTIVIDADES mediante una clave foránea.

Actividad 10. Eliminar la tabla CONTACTOS

```
DROP TABLE CONTACTOS;
```

Esta sentencia elimina la tabla y sus datos. Debe ejecutarse en un entorno de pruebas y después de retirar las dependencias correspondientes.

3. Actividad 11. Modificación de PruebaMySQL.java

El objetivo es mostrar exclusivamente los contactos cuya edad sea estrictamente superior al valor recibido como argumento. Por ejemplo, con una edad mínima de 30, se muestran los contactos de 31 años o más.

3.1. Código Java propuesto

```
/*
 * PruebaMySQL.java
 *
 * Programa de prueba para conectar con MySQL y consultar
 * los contactos que superen una edad determinada.
 *
 * Histórico:
 * Versión 1.1 - Octubre de 2026
 * Modificación: parámetro edadMinima y consulta parametrizada.
 * Mejora: cierre automático de recursos JDBC.
 */
package chuidiang.ejemplos.base_datos;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

/**
 * Consulta los contactos cuya edad es superior a edadMinima.
 */
public class PruebaMySQL {

   private static final String URL =
         "jdbc:mysql://localhost:3306/prueba";
```

```
private static final String USUARIO = "root";
private static final String CLAVE = "la_clave";

/**
 * Muestra los contactos cuya edad es superior a edadMinima.
 *
 * @param edadMinima edad de referencia; solo se muestran
 *                   contactos con edad estrictamente superior
 */
public PruebaMySQL(int edadMinima) {

   String sql = "SELECT Id, Nombre, Apellidos, Edad "
           + "FROM CONTACTOS WHERE Edad > ? "
           + "ORDER BY Id";

   try {
      Class.forName("com.mysql.cj.jdbc.Driver");

      try (Connection conexion = DriverManager.getConnection(
               URL, USUARIO, CLAVE);
           PreparedStatement ps =
               conexion.prepareStatement(sql)) {

         ps.setInt(1, edadMinima);

         try (ResultSet rs = ps.executeQuery()) {
            System.out.println(
                  "Contactos con edad superior a "
                  + edadMinima + ":");

            while (rs.next()) {
```

```
                System.out.println(
                        rs.getInt("Id") + " "
                        + rs.getString("Nombre") + " "
                        + rs.getString("Apellidos") + " "
                        + rs.getInt("Edad"));
                }
              }
            }

        } catch (ClassNotFoundException e) {
            System.err.println(
                    "No se ha encontrado el controlador JDBC de MySQL.");
            e.printStackTrace();

        } catch (SQLException e) {
            System.err.println(
                    "Error al conectar o consultar la base de datos.");
            e.printStackTrace();
        }
    }

    /**
     * Punto de entrada del programa.
     *
     * @param args primer argumento: edad mínima numérica
     */
    public static void main(String[] args) {

        if (args.length != 1) {
            System.out.println("Uso: PruebaMySQL <edadMinima>");
            return;
```

```
    }

    try {
      int edadMinima = Integer.parseInt(args[0]);

      if (edadMinima < 0) {
        System.out.println(
            "La edad mínima no puede ser negativa.");
        return;
      }

      new PruebaMySQL(edadMinima);

    } catch (NumberFormatException e) {
      System.out.println(
          "Error: debes introducir una edad numérica.");
    }
  }
}
```

Configuración necesaria: ajustar URL, usuario y contraseña a la base de datos real. El ejemplo utiliza la base prueba indicada en el enunciado; el proyecto Docker utiliza demo_db. El controlador com.mysql.cj.jdbc.Driver debe estar disponible como dependencia JDBC en tiempo de ejecución.

3.2. Resumen de las modificaciones

Elemento	Cambio	Finalidad
Constructor	PruebaMySQL(int edadMinima)	Recibir la edad de referencia.
Consulta SQL	WHERE Edad > ?	Filtrar contactos por edad.
PreparedStatement	ps.setInt(1, edadMinima)	Pasar el parámetro de forma segura.

ResultSet	Leer columnas por nombre	Mostrar datos de cada contacto.
Recursos JDBC	try-with-resources	Cerrar automáticamente los recursos.
main	Validación de args	Evitar parámetros ausentes o no numéricos.
Comentarios	Histórico y JavaDoc	Documentar el cambio.

3.3. Ejecución esperada

Con los datos de ejemplo, al ejecutar el programa con el argumento 30:

Contactos con edad superior a 30:

1 Alejandro Pérez López 35

Raimundo no aparece porque tiene 30 años y la condición SQL utiliza >, no >=.

4. Pruebas de la clase

Se recomienda combinar pruebas unitarias de la lógica de filtrado con pruebas de integración sobre una base de datos MySQL de pruebas. Las pruebas deben ejecutarse y documentarse antes de afirmar que la modificación está validada.

Prueba	Condición	Resultado esperado
P01	Edad mínima 30	Solo aparece Alejandro, de 35 años.
P02	Edad mínima 35	No aparecen contactos.
P03	Edad mínima 29	Aparecen Alejandro y Raimundo.
P04	Argumento abc	Mensaje de error; no se ejecuta la consulta.
P05	Argumento negativo	Se rechaza el valor.
P06	MySQL no disponible	Se gestiona el error de conexión.
P07	Finalización de consulta	Los recursos JDBC se cierran.

4.1. Prueba unitaria ilustrativa con JUnit 5

Para probar la lógica de filtrado de forma aislada, conviene separarla del acceso a datos mediante un servicio o clase auxiliar. El siguiente ejemplo presupone una clase

FiltroContactos con el método filtrarMayoresQue y un modelo Contacto con constructor y getters compatibles. Es una propuesta que debe implementarse antes de ejecutarla.

```
import static org.junit.jupiter.api.Assertions.*;
import java.util.List;
import org.junit.jupiter.api.Test;

class FiltroContactosTest {

  @Test
  void debeDevolverSoloContactosConEdadSuperior() {
    List<Contacto> contactos = List.of(
      new Contacto(1, "Alejandro", "Pérez López", 35),
      new Contacto(2, "Raimundo", "Guzmán Álvarez", 30)
    );

    List<Contacto> resultado =
      FiltroContactos.filtrarMayoresQue(contactos, 30);

    assertEquals(1, resultado.size());
    assertEquals("Alejandro", resultado.get(0).getNombre());
    assertTrue(resultado.get(0).getEdad() > 30);
  }
}
```

Además, una prueba de integración debe insertar los contactos de ejemplo en una base de datos de pruebas y verificar que la consulta WHERE Edad > ? devuelve los registros correctos. Para ejecutar el test será necesario añadir JUnit 5 al pom.xml y crear la clase auxiliar de filtrado.

5. Relación entre los requisitos y el proyecto

Requisito	Estado / evidencia

Tabla con clave primaria autoincremental	Implementado en la inicialización de ContactoDAO; verificar definición exacta.
Inserción de contactos de ejemplo	Implementado cuando la tabla está vacía.
Acceso a MySQL mediante JDBC	Implementado en ContactoDAO.
Consultas parametrizadas	Implementado en agregar() mediante PreparedStatement.
Filtro de contactos por edad	Pendiente de incorporar a la aplicación real.
Tabla ACTIVIDADES y relación	Resuelto en las consultas SQL de esta memoria; pendiente de integrar.
Índice único	Resuelto en SQL; no integrado en la aplicación.
Pruebas automatizadas	Pendientes de implementar y ejecutar.
Documentación e histórico	Propuestos en esta memoria y en los comentarios del código.

6. Conclusiones

La actividad permite demostrar la capacidad para crear y modificar tablas relacionales, insertar datos, establecer relaciones mediante claves foráneas y gestionar restricciones de integridad e índices.

También se propone la modificación de una clase Java para incorporar un parámetro que condiciona la consulta SQL, utilizando JDBC y PreparedStatement. El proyecto Gestor de Contactos complementa estos conocimientos mediante una arquitectura por capas que separa el modelo, el acceso a datos, el controlador y la interfaz.

Para completar la validación, deben ejecutarse pruebas unitarias sobre la lógica de filtrado y pruebas de integración contra MySQL, registrando sus resultados. La entrega relaciona los conocimientos de SQL y JDBC con una aplicación práctica y con los criterios CE2.6 del módulo formativo UF2405.